INTERMEDIATE SERIES · GUIDE 5 OF 8

Intermediate CrewAI

Custom tools, knowledge sources, memory, flows, testing, production patterns

01 Custom Tools — Deep Dive

While CrewAI has 20+ built-in tools, production systems need custom tools that connect to your own APIs, databases, and services. Two ways: BaseTool class or @tool decorator.

```
# Method 1: @tool decorator (simple)
from crewai.tools import tool

@tool("Customer Database Lookup")
def lookup_customer(customer_id: str) -> str:
    """Look up customer details from CRM by customer ID.
    Use when you need customer name, email, status, or history.
    """
    customer = crm_db.query(f"SELECT * FROM customers WHERE id={customer_id!
r}")
    if not customer:
        return f"No customer found with ID {customer_id}"
    return f"Name: {customer.name}, Email: {customer.email}, Status: {custom
er.status}"
```

```
# Method 2: BaseTool class (full control)
from crewai.tools import BaseTool
from pydantic import BaseModel, Field

class KYCCheckInput(BaseModel):
    pan_number: str = Field(description="PAN card number to verify")
    dob: str = Field(description="Date of birth in YYYY-MM-DD format")

class KYCVerificationTool(BaseTool):
    name: str = "KYC Verification"
    description: str = ("""Verify customer PAN and date of birth against
        government database. Returns VERIFIED, MISMATCH, or NOT_FOUND.""")
    args_schema: type[BaseModel] = KYCCheckInput

    def _run(self, pan_number: str, dob: str) -> str:
        try:
            result = kyc_api.verify(pan=pan_number, dob=dob)
            return f"Status: {result.status}, Name: {result.matched_name}"
```

```
        except APIError as e:
            return f"Verification failed: {e}"

    async def _arun(self, pan_number: str, dob: str) -> str:
        result = await kyc_api.async_verify(pan=pan_number, dob=dob)
        return f"Status: {result.status}"
```

02 Knowledge Sources — RAG Inside CrewAI

CrewAI Knowledge allows agents to have a RAG knowledge base without needing a separate tool call. Agents automatically retrieve relevant context when processing tasks.

```
# CrewAI Knowledge with PDF and text sources
from crewai import Agent, Task, Crew, Knowledge
from crewai.knowledge.source import PDFKnowledgeSource, TextKnowledgeSource

# Define knowledge sources
policy_docs = PDFKnowledgeSource(
    file_paths=["company_policy.pdf", "compliance_rules.pdf"])

faq_text = TextKnowledgeSource(
    content="""
    Q: What documents are required for KYC?
    A: PAN card, Aadhaar, and bank statement...
    """)

# Assign knowledge to specific agent
compliance_agent = Agent(
    role="Compliance Officer",
    goal="Ensure all customer onboarding meets regulatory standards",
    backstory="Expert in RBI compliance and KYC regulations",
    knowledge_sources=[policy_docs, faq_text],
)

# Or assign to entire crew
crew = Crew(
    agents=[compliance_agent],
    tasks=[review_task],
    knowledge_sources=[policy_docs]  # all agents can access
)
```

03 Memory Configuration

```
# Enable all memory types
from crewai import Crew, Process
from crewai.memory import (LongTermMemory, ShortTermMemory,
                           EntityMemory, ExternalMemory)
from crewai.memory.storage import LTMSQLiteStorage
```

```
crew = Crew(
    agents=[researcher, analyst],
    tasks=[research_task, analysis_task],
    process=Process.sequential,

    # Memory configuration
    memory=True,  # enables short-term + entity memory

    long_term_memory=LongTermMemory(
        storage=LTMSQLiteStorage("./crew_memory.db")  # persists to disk
    ),

    # Short-term: within this crew run
    short_term_memory=ShortTermMemory()
)

# Reset memory when needed
crew.reset_memories(command_type="all")
```

04 CrewAI Flows — Event-Driven Pipelines

Flows add conditional routing, state management, and event-driven orchestration on top of crews. A Flow can combine multiple crews with Python logic between them.

```
# Production KYC Flow with routing
from crewai.flow.flow import Flow, start, listen, router
from pydantic import BaseModel

class KYCState(BaseModel):
    customer_id: str = ""
    documents: list = []
    verification_result: str = ""
    risk_score: float = 0.0

class KYCFlow(Flow[KYCState]):

    @start()
    def collect_documents(self):
        """Crew 1: Document collection and OCR."""
        result = document_crew.kickoff(
            inputs={"customer_id": self.state.customer_id})
        self.state.documents = result.raw

    @listen(collect_documents)
    @router()
    def check_completeness(self):
        """Route based on document completeness."""
        required = ["pan", "aadhaar", "bank_statement"]
        found = [d["type"] for d in self.state.documents]
```

```
        missing = [r for r in required if r not in found]
        if missing:
            return "request_more_docs"
        return "verify_documents"

    @listen("verify_documents")
    def run_verification(self):
        """Crew 2: KYC verification."""
        result = verification_crew.kickoff(
            inputs={"documents": self.state.documents})
        self.state.verification_result = result.raw

    @listen("request_more_docs")
    def notify_incomplete(self):
        notify_customer(self.state.customer_id, "Additional docs required")

# Run the flow
flow = KYCFlow()
flow.kickoff(inputs={"customer_id": "CUST_001"})
```

05 Testing and Debugging CrewAI

```
# Unit test individual agents and tasks
from crewai import Agent, Task, Crew
import pytest

def test_researcher_uses_search_tool():
    """Verify researcher uses search tool for factual queries."""
    agent = Agent(
        role="Researcher",
        goal="Find accurate information",
        tools=[mock_search_tool],
        llm="gpt-4o-mini")

    task = Task(
        description="Find the current RBI repo rate.",
        expected_output="A specific percentage with source.",
        agent=agent)

    crew = Crew(agents=[agent], tasks=[task])
    result = crew.kickoff()

    assert "%" in result.raw  # should contain a percentage
    assert mock_search_tool.was_called  # should have searched
```

```
# Enable verbose logging for debugging
crew = Crew(
    agents=[researcher],
    tasks=[task],
```

```
    verbose=True,   # print every agent thought and tool call
)

# Or use CrewAI telemetry (opt-in)
# Set environment variable: CREWAI_TELEMETRY=false to disable
```